

Gestionale Weiss

Analisi approfondita dello stato del progetto

Problemi rilevati e soluzioni proposte

Progetto: weiss-gestionale — sistema gestionale per bar/cocktail bar (Weiss Cafè, Sacile)

Repository: github.com/NicScarpa/weiss-gestionale

Data analisi: 5 agosto 2026

Metodo: analisi incrociata di documentazione PRD, codice sorgente (562 file TS, ~114.000 righe), schema dati (65 modelli Prisma), audit di sicurezza e cronologia git

Indice

1. Executive Summary
2. Panoramica del progetto e stack tecnologico
3. La documentazione PRD: visione e lacune
4. Stato di implementazione modulo per modulo
5. Criticità di sicurezza — problemi e soluzioni
6. Criticità architetturali — problemi e soluzioni
7. Difetti funzionali per modulo — problemi e soluzioni
8. Analisi strategica: build vs buy e soluzioni alternative
9. Roadmap consigliata

1. Executive Summary

~114k

RIGHE DI CODICE IN
SRC/

184

ROUTE API

65

MODELLI PRISMA

~80%

REQUISITI PRD
COSTRUITI

Il progetto è in uno stato molto più avanzato di quanto sembri dall'esterno: i moduli core (chiusura cassa, generazione turni con solver reale, report, anagrafiche) sono completi e funzionanti, senza dati finti. Il problema dominante non è codice mancante, ma **codice scritto e mai collegato**: la pagina Budget funzionante è irraggiungibile dalla navigazione, la catena delle notifiche push non consegna nulla, circa 20 API complete non hanno alcun consumer.

Tre urgenze precedono qualsiasi nuovo sviluppo:

1. **Credenziali reali esposte nella history di GitHub** (Agenzia delle Entrate, Sibill, Fluida, Sesame) — richiedono rotazione immediata;
2. **ENCRYPTION_KEY assente**, che fa fallire a runtime i salvataggi di IBAN e codici fiscali, più tre bug di design nella cifratura dei campi;
3. **93 file di remediation di sicurezza mai committati** — il lavoro fatto non è protetto da nulla.

Verdetto architetturale: non serve riscrivere né cambiare stack. Next.js + Prisma + PostgreSQL con tipi `Decimal` per gli importi è una base corretta. Serve una fase di **consolidamento** (2–3 settimane di lavoro mirato) per passare da "80% costruito" a "80% utilizzabile" — due condizioni che oggi non coincidono.

Aggiornamento 5 agosto 2026: il provider **A-Cube** è stato validato con un test completo in sandbox (§8.6): open banking e fatturazione elettronica funzionano end-to-end e producono esattamente i dati che il gestionale si aspetta. La decisione build vs buy delle integrazioni SDI e bancarie è quindi risolta: si compra.

2. Panoramica del progetto e stack tecnologico

Weiss Café è un cocktail bar con caffetteria a Sacile (PN), con tre fasce operative (mattina/caffetteria, aperitivo 16–21, sera/cocktail bar) ed estensione prevista a due sedi secondarie (Villa Varda, Casette). Il gestionale nasce per sostituire un file Excel da ~30 fogli e il modulo cartaceo di chiusura cassa ridigitato a mano il giorno dopo.

Livello	Tecnologia	Note
Frontend	Next.js 16.1.6 (App Router), React 19.2, TypeScript	PWA via Servist, offline con IndexedDB
UI	Radix/shadcn, Tailwind 4, TanStack Query, react-hook-form + Zod 4	—
Backend	Route API Next.js, Prisma 7.2	Nessun service layer (v. §6.1)
Database	PostgreSQL su Supabase	Importi in <code>Decimal(10,2)</code> — scelta corretta
Auth	NextAuth v5 beta, Credentials + JWT	Sessione 8h, ricontrollo utente in DB
Servizi	Sentry, Upstash Redis (non configurato), Resend, firebase-admin	Push mai collegato lato client (v. §7.9)
Documenti	@react-pdf/renderer, jsPDF, pdf-lib, pdf-parse, exceljs, fast-xml-parser	Parser FatturaPA e P7M proprietari
Test / CI	Vitest, Playwright, Husky, GitHub Actions	Copertura concentrata su chiusure e prima nota
Deploy	Vercel (1 cron: auto-clockout)	Upload su filesystem locale incompatibili (v. §6.2)

Ruoli e utenti

Tre ruoli piatti: **Admin** (titolare, accesso completo), **Manager** (inserimento e report, potere pieno solo sul perimetro Staff), **Staff** (compila la chiusura cassa, usa il portale dipendente). A questi si aggiunge la distinzione operativa tra dipendenti fissi ed "extra" (collaboratori occasionali).

3. La documentazione PRD: visione e lacune

3.1 Documenti presenti

Documento	Contenuto
PRD/PRD_v1_1.md	PRD principale (superset della v1.0): chiusura cassa, prima nota, report, budget, SDI, riconciliazione, scadenzario. Svolta mobile-first/offline-first rispetto alla v1.0.
PRD/PRD_Modulo_Gestione_Personale_v1.0.md	1.517 righe: anagrafica avanzata, vincoli relazionali sui turni (il differenziatore dichiarato vs JetHR/Sesame), solver turni, portale dipendente PWA, presenze con GPS e QR rotante.
PRD/Budget_Section_Specifications.md	839 righe: categorie 100% configurabili dall'utente (mai hardcoded), 6 KPI di sistema, mapping conti→categorie drag & drop.
PRD/SPEC_Gestione_Utenti.md	2.187 righe: matrice permessi, username auto-generati, flusso password iniziale.
docs/Analisi_PRD_vs_Implementazione_2026-01-06.md	Gap analysis di gennaio: copertura stimata 78–82%.
docs/Soluzioni_Timbratura_Dipendenti.md	Confronto di 8 soluzioni di timbratura; raccomanda ibrido QR+GPS. Non ancora eseguito.
modifiche/	5 coppie piano+prompt non ancora implementate (inbox avvisi, azioni rapide, fix chiusura cassa, pagamento personale, PDF chiusura).

3.2 Lacune della documentazione stessa

P-DOC-1 — GDPR e privacy mai citati nei PRD ALTA

PROBLEMA

Il sistema tratta codici fiscali, retribuzioni, presenze, coordinate GPS, indirizzi IP e device ID, ma i PRD non menzionano mai GDPR, informative o basi giuridiche. L'unico documento che affronta il tema (correttamente) è `Soluzioni_Timbratura_Dipendenti.md`.

SOLUZIONE PROPOSTA

Redigere un allegato privacy unico che copra: informativa dipendenti (base giuridica: esecuzione del contratto), registro dei trattamenti, conservazione presenze 5 anni, accordo sindacale o autorizzazione ITL ex art. 4 Statuto dei Lavoratori prima di attivare geolocalizzazione/QR (obbligatorio, non opzionale). Evitare qualsiasi biometria: il documento timbrature la sconsiglia già per costi e DPIA obbligatoria.

P-DOC-2 — Conformità fiscale trattata in modo implicito MEDIA

PROBLEMA

I PRD gestiscono corrispettivi, IVA e fatture SDI ma non citano mai: conservazione sostitutiva a norma (l'XML è salvato in un campo TEXT senza requisiti di conservazione decennale), corrispettivi telematici/registratore telematico, libri obbligatori. L'impianto dare/avere è una prima nota a doppia sezione, non una contabilità generale (mancano bilancio, chiusura d'esercizio, ammortamenti) — il che è accettabile per lo scopo, ma va dichiarato.

SOLUZIONE PROPOSTA

Dichiarare esplicitamente nel PRD il perimetro: "prima nota gestionale, non contabilità civilistica — la contabilità ufficiale resta al commercialista". Per la conservazione a norma delle fatture, delegarla al provider SDI (v. §8.1): tutti i provider commerciali la includono.

P-DOC-3 — Incoerenze tra spec da riconciliare MEDIA

PROBLEMA

(a) PRD Personale vs SPEC Utenti divergono su tipi contratto (fisso/extra/intermittente vs FIXED/EXTRA), tariffe orarie (4 differenziate vs 1 sola), autenticazione staff (PIN vs username+password) e multi-sede (assunto dal primo, escluso dalla seconda). (b) La spec Budget contraddice sé stessa sul segno degli importi: la vista SQL usa `credit - debit` per tutte le categorie, le funzioni di calcolo usano `debit` per i ricavi e `credit` per i costi. (c) La spec Budget non definisce alcun controllo di autorizzazione.

SOLUZIONE PROPOSTA

Una sessione di riconciliazione delle spec (mezza giornata) prima di sviluppare il modulo personale
Fase 4: decidere multi-sede sì/no (v. P-ARC-2), una sola fonte di verità per i tipi contratto, e fissare la convenzione dei segni: ricavi = `credit - debit`, costi = `debit - credit`, sempre.

4. Stato di implementazione modulo per modulo

Modulo	Stato	Sintesi
Chiusura cassa	COMPLETO	Workflow DRAFT→SUBMITTED→VALIDATED, conteggio banconote per stazione, export PDF/Excel, bulk delete. Nessuna lacuna funzionale.
Turni	COMPLETO	Solver reale ~1.620 righe (greedy + local search), vincoli hard/soft, drag&drop, publish, export. 3 vincoli relazionali su 5 non implementati (v. §7.6).
Report	COMPLETO	4 report funzionanti con dati reali. Solo export CSV.
Staff / Anagrafiche / Prodotti / Auth / Scambi turno	COMPLETI	CRUD completo, inviti con token, storico prezzi. Lacune minori.
Prima nota	PARZIALE	CRUD e import ok; client con export e saldi è orfano; ordinamento morto; stato "Approva" invertito (v. §7.2).
Scadenzario	PARZIALE	Lista, calendario, aging, ricorrenze ok; le "Regole" sono CRUD senza motore (v. §7.4).
Fatture	PARZIALE	Ricevute complete con import XML/P7M; Emesse e Corrispettivi stub; 3 tab in 404 (v. §7.5).
Riconciliazione	PARZIALE	Matching reale con audit; mancano paginazione, filtri data, export.
Presenze / Timbratura	PARZIALE	Geofencing e coda offline reali, ma 3 difetti seri sull'affidabilità del dato (v. §7.7).
Ferie / Permessi	PARZIALE	Flusso completo, ma manca la UI del monte ferie e le card saldo sono sempre vuote (v. §7.8).
Documenti dipendenti	PARZIALE	Split cedolini PDF funzionante; pagine non abbinate scartate; storage locale (v. §6.2).
Budget	SCOLLEGATO	Le pagine Budget funzionano ma sono irraggiungibili: la route <code>/budget</code> mostra il Cash Flow (v. §7.1).
Notifiche	CATENA ROTTA	Server FCM pronto, client mai collegato: nessuna notifica viene consegnata (v. §7.9).

Test e CI

- **Vitest:** 10 file, ~4.660 righe — ottima copertura su parser SDI, chiusure, calcoli contabili, prima nota.
- **Playwright:** 11 spec, ~2.050 righe — prima nota, chiusura cassa, timbratura (anche offline), fatture, auth.

- **Senza alcun test:** budget, scadenziario, riconciliazione, report, documenti dipendenti e — soprattutto — il solver turni da 1.620 righe.
- **CI incoerente:** `.github/workflows/ci.yml` dichiara Node 20, ma `package.json` richiede Node $\geq 22 < 23$. Da allineare.

5. Criticità di sicurezza — problemi e soluzioni

Esistono due audit interni (docs/Security_Audit_Report_2026-01-11.md , score 5.1/10, e SECURITY_AUDIT_REPORT.md del 21 febbraio). Gran parte delle criticità del secondo è stata risolta (fix IDOR verificati, security header completi, sessione a 8h, dipendenze aggiornate, magic bytes sugli upload, PII scrubbing su Sentry) — ma **tutte le modifiche sono in 93 file non committati**. Restano aperti i punti seguenti.

S-1 — Credenziali reali esposte nella history di GitHub

CRITICA — AGIRE SUBITO

PROBLEMA

credenziali.env e credenziali_fluida.env sono stati committati (commit d312163 , contenuto in origin/main sul repo pubblico) e rimossi dal tracking solo in seguito: git show d312163:credenziali.env restituisce ancora tutto. Contengono: **codice fiscale, password e PIN Fisconline/Entratel** (accesso al cassetto fiscale dell'Agenzia delle Entrate), credenziali **Sibill, Fluida e Sesame**. La stessa password è riusata su tre servizi.

PERCHÉ È GRAVE

Il repo è su GitHub da mesi: i bot che scandagliano i repository per segreti operano in minuti, non mesi. Va assunto che le credenziali siano compromesse.

SOLUZIONE — IN QUEST'ORDINE

1. **Ruotare subito tutte le credenziali:** password+PIN Fisconline (dal portale AdE), Sibill, Fluida, Sesame. Password uniche per servizio, gestite con un password manager.
2. Verificare gli accessi recenti sui portali (l'AdE mostra gli ultimi accessi al cassetto fiscale).
3. Ripulire la history con git filter-repo (o BFG) e force-push — utile ma *non sufficiente* da sola: i commit possono essere già stati clonati.
4. Aggiungere gitLeaks come hook pre-commit (Husky è già configurato) per impedire il ripetersi.
5. Spostare i segreti in .env.local (già ignorato) o nelle env di Vercel.

S-2 — 93 file di remediation non committati

CRITICA

PROBLEMA

Tutto il lavoro di sicurezza post-audit (fix IDOR, header, sessioni, aggiornamenti) vive solo nella working directory: un git checkout sbagliato o un guasto del disco lo cancella, e il deploy in produzione non lo include.

SOLUZIONE

Commit immediato, spezzato per tema (sicurezza / feature / config) per mantenere la history leggibile. Prima del commit: verificare con git status che nessun file di credenziali o upload sia incluso, e aggiungere uploads/ a .gitignore (oggi manca: i cedolini rischiano di finire committati).

S-3 — Rate limiting assente sul login **ALTA**

PROBLEMA

Il rate limiting è applicato a 8 route su 184, ma `authorize()` in `src/lib/auth.ts` non ne ha: il brute force sul login è possibile. Inoltre `UPSTASH_REDIS_REST_URL/TOKEN` non sono configurati, quindi `src/lib/rate-limit.ts` usa il fallback in-memory — che su Vercel serverless non condivide stato tra istanze ed è di fatto inefficace anche dove applicato.

SOLUZIONE

(1) Creare il database Upstash Redis (piano gratuito sufficiente) e impostare le due variabili. (2) Aggiungere in testa ad `authorize()` un check per IP+username (es. 5 tentativi / 15 minuti). (3) In alternativa o in aggiunta, implementare il blocco account dopo 5 tentativi falliti già previsto dalla SPEC Utenti (campo contatore su `User` + reset al login riuscito).

S-4 — API in lettura senza controllo di ruolo **ALTA**

PROBLEMA

In `src/middleware.ts:5` l'array `STAFF_ALLOWED_PREFIXES` è definito ma mai usato. Le *pagine* sono protette (il layout reindirizza lo staff al portale), ma le *API* no: un utente staff autentificato può chiamare direttamente `GET /api/prima-nota`, `/api/scadenzario`, `/api/pagamenti`, `/api/budget/confronto` e i quattro report, vedendo dati finanziari che non gli competono. Le mutazioni sono quasi sempre protette, ma con controlli copiati a mano in ~60 file; l'helper `requireRole` (`src/lib/api-utils.ts:137`) è usato in una sola route.

SOLUZIONE

Due interventi complementari: (1) attivare `STAFF_ALLOWED_PREFIXES` nel middleware come prima barriera per le route `/api/*`; (2) sostituire progressivamente i confronti manuali con `requireRole(['admin', 'manager'])` — iniziando dalle GET finanziarie scoperte. Vedi anche P-ARC-6 sul modello permessi.

S-5 — CSP con unsafe-inline e unsafe-eval **MEDIA**

PROBLEMA

In `next.config.ts:26`, `script-src 'self' 'unsafe-inline' 'unsafe-eval'` rende la Content-Security-Policy quasi inefficace come mitigazione XSS, in un'app che mostra IBAN e dati retributivi.

SOLUZIONE

Passare a CSP con nonce (Next.js la supporta via middleware: generazione nonce per request e propagazione negli script). `unsafe-eval` in produzione di solito non serve: verificare se lo richiede solo il dev mode e differenziare la policy per ambiente.

S-6 — Middleware senza verifica crittografica del token BASSA — ACCETTATA

PROBLEMA

`src/middleware.ts:60-68` verifica solo la *presenza* del cookie di sessione, non firma né scadenza (NextAuth v5 usa JWE non verificabile in edge runtime senza chiave). La validazione reale è delegata ad `auth()` in pagine e API.

SOLUZIONE

Scelta consapevole e documentata nel codice: accettabile. Da tenere presente come vincolo: nessuna route API deve mai fidarsi del solo passaggio dal middleware. La regola è già rispettata (solo 5 route su 184 non chiamano `auth()`, tutte intenzionalmente pubbliche).

6. Criticità architetture — problemi e soluzioni

P-ARC-1 — ENCRYPTION_KEY assente + cifratura dei campi difettosa CRITICA — BUG ATTIVI

PROBLEMA

`src/lib/encryption.ts:11` lancia un errore se `ENCRYPTION_KEY` manca — e in `.env` manca. Poiché `fieldEncryptionExtension` è montata su ogni client Prisma, qualsiasi `create/update` che tocchi `Supplier.iban`, `User.fiscalCode`, `BankAccount.iban` ecc. esplode a runtime. In più il design ha tre difetti:

- Le clausole `where` non vengono cifrate: `src/lib/sdi/matcher.ts:94` cerca i fornitori per `fiscalCode` in chiaro contro dati cifrati → non trova mai nulla → **fornitori duplicati a ogni import fattura** (bug silenzioso: il match per P.IVA regge perché quella non è cifrata).
- AES-256-GCM con IV casuale produce ciphertext diversi per lo stesso valore: il vincolo `@unique([venueId, iban])` su `BankAccount` è neutralizzato.
- `decryptResult` decifra solo il record di primo livello: gli `include` annidati (es. scadenziario → fornitore) restituiscono IBAN e CF ancora cifrati al client.

SOLUZIONE

1. Generare e impostare subito `ENCRYPTION_KEY` (32 byte, `openssl rand -hex 32`) in `.env.local` e nelle env di Vercel.
2. **Ridimensionare la cifratura applicativa:** Supabase cifra già a riposo; oggi il costo in bug supera il beneficio. Consiglio: mantenere la cifratura solo sui campi mai usati in ricerca (es. note sensibili), e per i campi ricercabili (codice fiscale, IBAN) affiancare una *colonna hash di lookup* (HMAC-SHA256 con chiave dedicata) su cui fare match e vincoli unique, oppure passare a cifratura deterministica per quei soli campi.
3. Nel frattempo, fix puntuale del matcher: cercare i fornitori tramite la colonna hash (o per P.IVA con fallback), mai per campo cifrato.
4. Estendere `decryptResult` alle relazioni incluse, o meglio centralizzare la decifratura in un service layer (v. P-ARC-4).

P-ARC-2 — Multi-tenancy solo nominale CRITICA AL MOMENTO DELLA 2ª SEDE

PROBLEMA

Ogni modello ha `venueId` e le API filtrano su di esso, ma `src/lib/venue.ts` risolve la sede con `findFirst({ where: { isActive: true } })` — "la prima sede attiva" — ignorando `session.user.venueId` (zero occorrenze in 184 route). I controlli tipo `if (closure.venueId !== venueId)` `return 403` confrontano la risorsa con l'unica sede esistente: sono tautologici. Il giorno in cui Villa Varda diventasse una seconda venue attiva, `getVenueId()` restituirebbe una sede *arbitraria* e l'intero lavoro anti-IDOR diventerebbe inefficace in blocco. Tre punti prendono già oggi "la prima sede" invece di quella giusta: `RiconciliazioneClient.tsx:28`, `budget/confronto/page.tsx:18`, la costruzione sede in `portale/timbra`.

SOLUZIONE

Decidere ora, senza vie di mezzo:

- **Opzione A (consigliata finché la sede è una):** congelare il multi-tenant. Documentare che l'app è single-venue, mantenere `venueId` nello schema ma vietare la creazione di una seconda venue attiva (vincolo o guard nel seed/API).
- **Opzione B (se le sedi arrivano davvero):** mettere `venueId` nel token di sessione al login (o un selettore sede per gli admin), riscrivere `getVenueId()` per leggerlo da lì, e correggere i tre punti già difettosi. Un pomeriggio di lavoro, ma va fatto *prima* della seconda sede, non dopo.

P-ARC-3 — Upload su filesystem locale, incompatibile con Vercel

ALTA — BLOCCA IL DEPLOY DEI DOCUMENTI

PROBLEMA

`documents/route.ts`, `documents/upload-bulk`, `scadenzario/[id]/allegati` e `portal/documents/[id]` leggono/scrivono in `process.cwd()/uploads`. Su Vercel il filesystem è read-only fuori da `/tmp` ed effimero: cedolini e allegati falliscono in scrittura o spariscono al deploy successivo, lasciando record orfani (i modelli salvano solo `filename`, senza URL).

SOLUZIONE

Migrare a **Supabase Storage** (il DB è già Supabase: zero servizi nuovi, bucket privati con signed URL). Passi: (1) creare bucket `documents` e `attachments` privati; (2) sostituire `writeFile/readFile` con l'SDK Supabase in un modulo unico `src/lib/storage.ts` così i 4 punti di upload condividono la stessa implementazione; (3) salvare il path del bucket nel record; (4) servire i download con signed URL a scadenza breve (i cedolini sono dati retributivi). Alternativa equivalente: Vercel Blob.

P-ARC-4 — Nessun service layer: logica contabile nelle route ALTA — DEBITO STRUTTURALE

PROBLEMA

Le transazioni di partita doppia vivono inline dentro route da 589 righe (`api/chiusure/[id]/route.ts`) e componenti client da 800–1.100 righe. Conseguenze concrete: la logica è testabile solo via HTTP (ed è il motivo per cui solver, riconciliazione e budget non hanno test), la duplicazione cresce (l'array dei modelli soft-delete è copiato 3 volte identico in `src/lib/prisma.ts`), e ogni bugfix va replicato a mano.

SOLUZIONE

Refactoring incrementale, non big-bang: creare `src/lib/services/` ed estrarre *una funzione alla volta* partendo dalle tre a maggior valore — `closureService.validate()` (genera i movimenti), `primaNotaService.createEntry()` , `reconciliationService.match()` . Ogni estrazione porta con sé i suoi unit test. Le route diventano thin controller: `auth` → `zod` → `service` → risposta. Stesso pattern per i componenti oltre le 800 righe (estrarre hook e sottocomponenti).

P-ARC-5 — Nessun indice su `journal_entries` ALTA — COSTO DI FIX BASSISSIMO

PROBLEMA

Il modello centrale della prima nota non ha alcun `@@index` , mentre modelli meno trafficati ne hanno 3–5. Prisma non crea indici automatici sulle FK in PostgreSQL: le query per `venueId + date` — l'accesso più frequente dell'app — fanno sequential scan. Stessa situazione per `daily_expenses` , `cash_stations` , `accounts` .

SOLUZIONE

Aggiungere allo schema e migrare:

```
model JournalEntry {
  ...
  @@index([venueId, date])
  @@index([accountId])
  @@index([venueId, registerType, date])
}
```

Dieci minuti di lavoro, beneficio permanente. Ripetere per gli altri tre modelli citati.

P-ARC-6 — Modello permessi dead code + soft delete incoerente MEDIA

PROBLEMA

(a) Le tabelle `Permission / RolePermission` e le funzioni `hasPermission()` / `getUserPermissions()` hanno **zero chiamate** in tutto il codebase: l'autorizzazione reale è fatta con confronti di stringa in 59 file, e un eventuale ruolo custom non passerebbe alcun controllo. (b) Otto modelli finanziari hanno `deletedAt` e un'estensione Prisma che lo filtra — ma solo su `findMany/findFirst/count` (non su `findUnique/aggregate/groupBy`), e soprattutto le cancellazioni reali restano **hard delete**: il campo non viene mai valorizzato.

SOLUZIONE

(a) Per un bar con 3 ruoli, il RBAC granulare è over-engineering: **eliminare** le tabelle permessi e le funzioni morte, standardizzare su `requireRole`. Meno codice, meno ambiguità. (b) Per le scritture contabili l'inalterabilità non è opzionale: sostituire i `delete` su `JournalEntry`, `DailyClosure` e `Payment` con `update({ deletedAt: new Date() })`, estendere il filtro dell'estensione ad `aggregate/groupBy` (oggi i report includerebbero i cancellati), e deduplicare l'array dei modelli soft-delete in una costante unica.

P-ARC-7 — ~20 API orfane e duplicati italiano/inglese MEDIA — IGIENE DEL CODEBASE

PROBLEMA

Coppie duplicate (`/api/pagamenti` vs `/api/payments`; *quattro* varianti delle regole di categorizzazione: `categorization-rules`, `categorizzazione`, `categorizzazione-rules`, `regole-categorizzazione`) e ~20 endpoint completi senza alcun consumer (`prima-nota/versamento`, `cashflow/forecasts/*`, `documents/[id]/assign`, `POST /api/leave-balance`...). È il pattern tipico dello sviluppo AI-assistito senza consolidamento: ogni sessione ricrea invece di riusare.

SOLUZIONE

(1) Fissare una convenzione: **una sola lingua per le API** (consiglio l'inglese, già maggioritaria). (2) Un giro di pulizia: per ogni orfana, decidere se collegarla (alcune risolvono lacune reali: `documents/[id]/assign` risolve il problema dei cedolini non abbinati, `POST /leave-balance` quello del monte ferie) o cancellarla. (3) Annotare la convenzione in `CLAUDE.md` così le sessioni future la rispettano.

P-ARC-8 — Helper esistenti ma non adottati BASSA

PROBLEMA

`handleApiError` (in `api-utils.ts:96`) non è usato da nessuna route: la forma delle risposte d'errore è replicata a mano 175 volte. `src/lib/logger.ts:74` ha ancora `// TODO: Integrare Sentry` benché Sentry sia configurato.

SOLUZIONE

Adottare gli helper nelle route man mano che si toccano (non serve una campagna dedicata) e collegare il logger a `Sentry.captureException` nel ramo `error` — 10 righe.

7. Difetti funzionali per modulo — problemi e soluzioni

7.1 Budget — modulo funzionante ma irraggiungibile **IL MIGLIOR RAPPORTO VALORE/SFORZO**

PROBLEMA

`src/app/(dashboard)/budget/page.tsx` non è la pagina Budget: è una dashboard Cash Flow. Di conseguenza `BudgetList.tsx` (391 righe) è codice morto e le pagine `/budget/nuovo`, `/budget/[id]` e `/budget/confronto` — tutte e tre *funzionanti* — sono irraggiungibili dalla sidebar. I pulsanti "Indietro" riportano al Cash Flow. Nella dashboard Cash Flow, i controlli daily/weekly/monthly non fanno nulla (stato `grouping` mai letto) e la soglia di allarme è hardcoded a 5000.

SOLUZIONE

(1) Spostare il Cash Flow su una route propria (`/cash-flow`) con voce sidebar dedicata. (2) Ripristinare `/budget` con `BudgetList`. (3) Collegare lo stato `grouping` ai controlli e rendere la soglia configurabile (campo su `Venue` o impostazioni). Mezza giornata di lavoro per recuperare un intero modulo.

7.2 Prima nota — client orfano e stato pagamenti invertito **ALTA**

PROBLEMA

(a) `PrimaNotaClient.tsx` (489 righe, con export PDF/Excel/CSV, card saldi, riepilogo dare/avere) non è mai importato: la pagina attiva ha perso quelle funzioni benché l'API di export (309 righe, 4 formati) sia completa. (b) In `PagamentiClient.tsx:205` il pulsante "Approva" imposta `DA_APPROVARE` invece di `DISPOSTO`: il workflow non avanza mai. (c) L'ordinamento delle colonne non fa nulla (prop `onSort` mai passata), il filtro ricerca pagamenti non viene applicato ma conta nel badge, la voce "Categorizza" mostra "non implementata" mentre l'API esiste.

SOLUZIONE

(a) Confrontare il client orfano con quello attivo e reintegrare le sezioni mancanti (o sostituire il client attivo se l'orfano è un superset). (b) Fix a una riga: lo stato target di "Approva" è `DISPOSTO`. (c) Passare `onSort`, propagare la ricerca ai parametri dell'API, collegare "Categorizza" all'endpoint esistente. Aggiungere paginazione ai pagamenti.

7.3 Riconciliazione — limiti operativi **MEDIA**

PROBLEMA

`limit=100` hardcoded senza paginazione; nessun filtro data in UI benché l'API accetti `dateFrom/dateTo`; nessun export; `RiconciliazioneClient.tsx:28` usa `venues[0]`.

SOLUZIONE

Esporre i filtri data già supportati dall'API, aggiungere paginazione server-side (pattern già presente in chiusure e fatture: riusarlo), export CSV col medesimo pattern dei report, e sostituire `venues[0]` con la sede di sessione (dipende da P-ARC-2).

7.4 Scadenzario — regole senza motore e link rotti MEDIA

PROBLEMA

(a) Il sottomodulo "Regole" (reverse-engineered da Sibill) è CRUD completo con drag&drop, ma `prisma.scheduleRule` compare solo nelle route CRUD: **nessun codice le applica**. La UI dichiara "le regole sono applicate nell'ordine visualizzato" — non è vero. (b) I link dell'aging passano query param che la pagina scadenziario non legge: si atterra senza filtri. (c) `handlePayment` e `handleCreateSchedule` falliscono in silenzio, senza toast.

SOLUZIONE

(a) Decidere: o si implementa il motore (un modulo `src/lib/schedule-rules/engine.ts` invocato alla creazione della scadenza e all'import fatture: prima regola che matcha per tipo documento+pagamento vince, in ordine di priorità), oppure si nasconde la tab finché il motore non esiste. Una UI che promette automazioni inesistenti mina la fiducia nel dato. (b) Leggere i param con `useSearchParams` e inizializzare i filtri. (c) Aggiungere toast di successo/errore (pattern già usato ovunque).

7.5 Fatture — stub visibili e tab in 404 MEDIA

PROBLEMA

`fatture/emesse` e `fatture/corrispettivi` sono stub "coming soon" di 12 righe; tre tab del layout (`/bozze`, `/esterometro`, `/importa`) puntano a rotte inesistenti → 404; il filtro "Divisione" è un Select senza `value` né `onValueChange`; manca il download del documento originale (PDF/XML) dal dettaglio.

SOLUZIONE

Regola: **ciò che non esiste non si mostra**. Rimuovere dal layout le tre tab in 404 e il filtro morto; sostituire gli stub con un empty state onesto ("In arrivo con l'integrazione SDI") o toglierli dalla navigazione. Aggiungere il download dell'XML originale (è già salvato in DB: serve solo una route che lo serva con il content-type corretto). Le funzioni vere (emesse, corrispettivi) hanno senso solo dopo la decisione sul provider SDI (v. §8.1).

7.6 Turni — vincoli offerti ma ignorati dal solver MEDIA — MINA LA FIDUCIA

PROBLEMA

In `src/lib/shift-generation/constraints.ts:414-427`, tre dei cinque vincoli relazionali (`SAME_DAY_OFF`, `MIN_OVERLAP`, `MAX_TOGETHER`) sono case vuoti — ma la UI li offre tutti: l'utente li configura e il solver li ignora in silenzio. Erano il *differenziatore dichiarato* del PRD Personale rispetto a JetHR/Sesame. Inoltre: fallback tariffa hardcoded a 10 €/h che falsa la stima costi (`greedy-solver.ts:310`), match del turno preferito basato su stringhe nel nome ("MATTINA"/"SERA" — un turno "Brunch" non matcha nulla), `SKILL_REQUIRED` configurabile ma non gestito. Il solver da 1.620 righe non ha un solo test.

SOLUZIONE

(1) Nel breve: nascondere dalla UI i tre vincoli non implementati (onestà del prodotto). (2) Implementarli come vincoli soft nello scoring del greedy + local search: `SAME_DAY_OFF` e `MAX_TOGETHER` sono valutabili giorno per giorno, `MIN_OVERLAP` a fine settimana nella fase di local search. (3) Sostituire il match per nome con un campo strutturato `timeSlot` sulla definizione turno. (4) Prima di toccare il solver, scrivere i test: è logica pura, perfetta per vitest (fixture di staff+vincoli → assert sulle violazioni).

7.7 Timbratura — tre difetti sull'affidabilità del dato ALTA — IMPLICAZIONI GIUSLAVORISTICHE

PROBLEMA

1. **Il timestamp delle timbrature offline viene perso:** il client invia `offlineTimestamp`, ma lo schema zod di `punch/route.ts:11-18` non lo include (zod lo strippa) e la riga 120 forza `punchedAt: new Date()`. Una timbratura delle 08:00 sincronizzata alle 14:00 risulta fatta alle 14:00.
2. **Non si può timbrare senza turno pubblicato oggi:** la pagina costruisce la sede solo da `todayAssignment.venue`; senza turno (o senza coordinate sede) il pulsante è disabilitato. Un extra chiamato all'ultimo o un turno non ancora pubblicato = niente timbratura.
3. **Geofencing aggirabile:** il controllo server scatta solo se il client invia le coordinate, e il client in caso di errore GPS prosegue senza. Negare il permesso di geolocalizzazione bypassa il vincolo.

Inoltre: dei 9 tipi di anomalia a schema solo 2 vengono generati; le tolleranze della policy (ritardi, pause) sono salvabili ma nessuna logica le legge.

SOLUZIONE

(1) Aggiungere `offlineTimestamp` allo schema zod e usarlo come `punchedAt` quando presente, con validazione di ragionevolezza (non nel futuro, non oltre 24h nel passato) e flag `isOfflineSync` per l'audit. (2) Fallback di sede: selettore tra le sedi assegnate al dipendente, con anomalia `NO_SHIFT` generata automaticamente per revisione del manager. (3) Se `requireGeolocation` è attivo per la sede, il server deve *rifiutare* la timbratura senza coordinate (o accettarla marcandola anomala) — la decisione va tolta al client. Completare la generazione delle anomalie leggendo le tolleranze della policy (`LATE_IN/EARLY_OUT` sono confronti banali col turno assegnato).

7.8 Ferie e permessi — monte ferie non gestibile MEDIA

PROBLEMA

(a) Non esiste UI per impostare il monte ferie: `POST /api/leave-balance` (completo, admin-only) non è chiamato da nessun componente → i saldi partono da 0 e vanno in negativo. (b) `LeaveTab.tsx:59` legge `balancesData?.balances` ma l'API risponde `{ data: [...] }` → le card saldo nel dettaglio dipendente sono *sempre vuote*. (c) Nessun CRUD per i tipi di assenza (solo GET). (d) Giornata parziale con 8 ore/giorno hardcoded, nessuna maturazione automatica del rateo.

SOLUZIONE

(a) Aggiungere nel dettaglio dipendente (tab ferie) un form admin che chiami l'API già esistente. (b) Fix a una riga sul nome del campo. (c) CRUD tipi assenza in impostazioni (pattern `AccountManagement` riusabile). (d) Maturazione: un cron mensile che accredita il rateo (per il CCNL pubblici esercizi tipicamente 26 giorni/anno per i full-time, pro-quota per i part-time) — oppure, più semplice e robusto per iniziare, l'inserimento manuale a inizio anno con il form del punto (a).

7.9 Notifiche — catena rotta end-to-end ALTA — IL PORTALE È MUTO

PROBLEMA

Il server FCM (`fcm.ts`) è completo, ma: il pacchetto client `firebase` non è in `package.json`, non esiste alcuna chiamata a `getToken()`, nessuna variabile `NEXT_PUBLIC_FIREBASE_*`, nessuna VAPID key, e `/api/notifications/subscribe` non è mai chiamato → la tabella `subscription` resta vuota e **nessuna notifica viene mai consegnata** (pubblicazione turni, ferie approvate, scambi turno, cedolini). Intanto `NotificationSettings.tsx` (290 righe) fa configurare preferenze per notifiche che non arriveranno; non esiste UI in-app che legga `/api/notifications/history`; `/api/shifts/reminder` è pronto ma non schedato in `vercel.json`; gli inviti dipendente non vengono mai spediti via email (l'azione bulk è un placeholder dichiarato).

SOLUZIONE

In ordine di resa:

1. **Prima la campanella in-app** (nessuna dipendenza esterna): un componente header che legge `/api/notifications/history` con polling TanStack Query. Rende utile da subito tutto il sistema già scritto.
2. Poi il push: installare `firebase` client, configurare il progetto FCM (VAPID key, env `NEXT_PUBLIC_FIREBASE_*`), chiamare `getToken()` nel service worker già presente (Serwist) e collegare `subscribe`.
3. Aggiungere `/api/shifts/reminder` ai cron di `vercel.json`.
4. Collegare l'invio email degli inviti via Resend (il template e il token esistono già; manca solo la chiamata).

7.10 Documenti dipendenti — pagine non abbinate scartate MEDIA

PROBLEMA

Nello split dei cedolini, le pagine il cui nominativo non matcha nessun dipendente vengono **scartate** (il buffer non viene persistito) senza possibilità di recupero; l'endpoint di riassegnazione manuale `PATCH /api/documents/[id]/assign` esiste ma è orfano; `pdf-splitter.ts:34` usa `eval('require')('pdf-parse')` per aggirare Turbopack (fragile in build standalone); la verifica del codice fiscale dichiara un esito ma non discrimina nulla.

SOLUZIONE

Salvare le pagine non abbinate come documenti "da assegnare" (stato `UNMATCHED`) e costruire la piccola UI di riassegnazione sopra l'endpoint già esistente. Sostituire l'`eval` con un `dynamic import()` marcato `serverExternalPackages` in `next.config.ts`. La migrazione dello storage (P-ARC-3) è prerequisito per l'uso in produzione.

8. Analisi strategica: build vs buy e soluzioni alternative

Da sviluppatore di gestionali, il principio guida per questo progetto è: **compra le integrazioni normative, costruisci il dominio**. Il valore differenziante — chiusura cassa modellata sul modulo cartaceo reale, solver turni con vincoli relazionali, prima nota sul flusso specifico del bar — nessun SaaS lo offre così. Le integrazioni con l'infrastruttura fiscale e bancaria italiana, invece, sono commodity che non conviene costruire in casa.

8.1 Fatturazione elettronica: provider API, non accreditamento diretto

Situazione: il parser FatturaPA/P7M proprietario (919+649 righe) è di buona qualità, ma l'import è manuale da file. Il PRD chiede la sincronizzazione automatica dal SDI, che richiederebbe accreditamento come intermediario — un percorso sproporzionato per un bar.

Soluzione consigliata: un provider API (A-Cube, Aruba Fatturazione, OpenAPI Fatture in Cloud). Per poche decine/centinaia di euro l'anno si ottengono: ricezione automatica delle passive via webhook o polling, trasmissione delle attive (sblocca la tab "Emesse" oggi stub), e **conservazione sostitutiva a norma inclusa** — che risolve anche la lacuna P-DOC-2. Il parser esistente resta utile come fallback e per l'estrazione dei prezzi fornitori. L'architettura a webhook si integra bene con le route esistenti.

8.2 Open Banking: aggregatore, non PSD2 diretta

Situazione: la PSD2 compare solo come stringhe nei tipi; la riconciliazione oggi importa CSV.

Soluzione consigliata: GoCardless Bank Account Data (ex Nordigen — piano gratuito fino a 50 connessioni, copre le banche italiane) o Fabrick per esigenze più spinte. L'integrazione è una route di callback + un job di sync movimenti che alimenta la stessa tabella `BankTransaction` già usata dall'import CSV: il motore di matching esistente non cambia. L'import CSV resta come fallback.

8.3 Presenze: decidere tra modulo interno e SaaS esistente

Situazione: esistono credenziali attive per Fluida e Sesame — segno che uno dei due è o era in uso. Il modulo interno è promettente (geofencing, coda offline) ma ha i tre difetti del §7.7, che sono anche un rischio giuslavoristico: un orario di timbratura sbagliato è un dato retributivo sbagliato.

Soluzione consigliata: decisione esplicita, non deriva. Se si tiene il modulo interno: correggere i tre difetti (2–3 giorni) e completare l'iter privacy (P-DOC-1) *prima* dell'uso reale; il vantaggio è l'integrazione nativa con turni e chiusura cassa (la sezione presenze della chiusura è già collegata). Se il tempo manca: mantenere il SaaS per la rilevazione legale e importarne i dati via export, usando il modulo interno solo in lettura. La cosa da evitare è il doppio binario non dichiarato.

8.4 Cosa NON cambiare

- **Stack:** Next.js + Prisma + PostgreSQL/Supabase è adeguato per scala e team. Nessuna riscrittura.
- **Modello dati contabile:** `Decimal` ovunque, piano dei conti gerarchico, dare/avere con contropartita, saldo progressivo denormalizzato — impianto corretto. Da sistemare solo indici (P-ARC-5) e soft delete (P-ARC-6).

- **Parser SDI/P7M:** proprietario ma testato (1.341 righe di test) e con dettagli non banali corretti (preservazione zeri iniziali nelle P.IVA). Tenerlo.
- **Solver turni:** l'approccio greedy + local search è giusto per questa scala (non serve un CSP solver completo). Va solo completato e testato.

8.5 Valutazione A-Cube — risultati del test sandbox (5 agosto 2026)

Il provider [A-Cube](#) (accreditato SDI, con API per fatturazione elettronica, cassetto fiscale, open banking, conservazione a norma e corrispettivi) è stato sottoposto a un **test tecnico completo in ambiente sandbox**, con account reale e flussi eseguiti end-to-end.

Cosa è stato verificato

Flusso	Esito	Dettaglio
Autenticazione	OK	Login → token JWT (24h) con ruoli su fatturazione IT, open banking, corrispettivi e conservazione.
Open Banking: connessione conto	OK	Anagrafica → <code>POST /connect</code> → link di consenso → widget con le principali banche italiane reali (Intesa, UniCredit, BPER, Poste, Fineco...) → consenso PSD2 → 5 conti collegati automaticamente .
Open Banking: transazioni	OK	Transazioni disponibili in pochi secondi con: data, importo, valuta, descrizione, IBAN e nominativo di controparte, saldo — e categorizzazione automatica con indice di confidenza (0.9 nel test).
Fatturazione: invio	OK	Fattura inviata come semplice JSON; A-Cube ha generato l'XML FatturaPA a norma e il simulatore SDI l'ha presa in carico con nome file ufficiale.
Fatturazione: ricezione passive	OK	Profilo con <code>supplier_invoice_enabled</code> ; webhook <code>supplier-invoice</code> alla consegna; P7M gestiti dal provider (XML estratto, originale scaricabile).
Conservazione / firma / corrispettivi	CONFIG.	Attivabili per profilo (<code>apply_legal_storage</code> , <code>apply_signature</code> , <code>receipts_enabled</code>), senza codice aggiuntivo.
Cassetto fiscale	NON TESTABILE	Il permesso è presente nel token ma il servizio non è raggiungibile in sandbox (richiede credenziali AdE reali). Da chiarire con il commerciale.

Scoperte rilevanti per la decisione

- **L'open banking poggia su Salt Edge**, aggregatore PSD2 tra i più consolidati in Europa: infrastruttura collaudata, non costruita in casa dal provider.
- **Consenso bancario: 90 giorni** (limite normativo PSD2; verificato sul dato reale). Ogni ~3 mesi va rinnovata l'autorizzazione — esiste l'endpoint dedicato `/accounts/{uuid}/reconnect` . Storico transazioni fino a 365 giorni.
- **I conti si abilitano singolarmente** (`enabled` per conto): probabile fatturazione per conto attivo — da confermare con il commerciale.

- Le transazioni si sincronizzano via **polling** (filtri `fetchAt` / `madeOn` fatti apposta); i webhook OB coprono connessione, riconnessione e pagamenti.
- La **categorizzazione automatica delle transazioni** inclusa nel servizio copre di fatto il requisito "categorizzazione AI" del PRD v1.1, mai realizzato in casa.
- La generazione dell'XML da JSON e la gestione P7M lato provider rendono ridondante parte del codice proprietario (`p7m-utils.ts`), che resta come fallback.

Compatibilità con l'assetto attuale (SDI T9K4ZHO via Datev)

Il codice destinatario registrato in AdE è unico per P.IVA: registrare quello di A-Cube sposterebbe la consegna delle fatture fuori dal canale Datev del commercialista. La configurazione senza interferenze è: **mantenere la consegna su T9K4ZHO** e usare A-Cube per il prelievo dal cassetto fiscale (con credenziali Fisconline dirette, che non consumano gli slot di delega: massimo 2 intermediari, e il commercialista ne occupa presumibilmente uno). Le deleghe convivono e nessuna revoca l'altra; la latenza aggiuntiva (le fatture compaiono nel cassetto dopo l'elaborazione SDI, in genere in giornata) è irrilevante per l'operatività di un bar.

Domande aperte per il commerciale

1. Prezzo per fattura (ricevuta/inviata) e per conto bancario attivo/mese; costo del piano minimo per una singola P.IVA
2. Conservazione a norma inclusa o a parte
3. Attivazione e prova del Cassetto Fiscale; modalità di accesso consigliata (credenziali dirette vs delega)
4. Promemoria automatico per il rinnovo del consenso PSD2 a 90 giorni

Verdetto: idoneo. Entrambi i flussi storicamente bloccanti (dati dal cassetto fiscale, estratto conto bancario) funzionano e producono dati direttamente mappabili sulle tabelle esistenti (`ElectronicInvoice` , `BankTransaction`); il motore di riconciliazione non richiede modifiche. Integrazione stimata in 1–2 settimane, con i prerequisiti già noti (rotazione credenziali S-1, fix matcher fornitori P-ARC-1).

8.6 Igiene dello sviluppo AI-assistito

Il pattern ricorrente del progetto (API duplicate in due lingue, client orfani, feature ricreate invece che riusate) è il costo tipico dello sviluppo con agenti AI senza regole di consolidamento. Tre contromisure a costo quasi nullo:

1. Annotare in `CLAUDE.md` le convenzioni: lingua unica per le API, "cerca prima di creare", helper obbligatori (`requireRole` , `handleApiError` , `getVenueId`).
2. Commit piccoli e frequenti — mai più 93 file in sospeso.
3. Ogni sessione di sviluppo termina con: il nuovo codice è raggiungibile dalla UI? Ha almeno un consumer? Se no, si collega o si cancella.

9. Roadmap consigliata

Fase 0 — Emergenza (1–2 giorni)

Azione	Riferimento	Sforzo
Rotazione credenziali AdE, Sibill, Fluida, Sesame + verifica accessi	S-1	2–3 ore
Commit delle 93 modifiche pendenti (spezzato per tema)	S-2	2–3 ore
Generare e impostare <code>ENCRYPTION_KEY</code> (locale + Vercel)	P-ARC-1	30 min
Pulizia history git + hook gitleaks	S-1	2 ore
<code>uploads/</code> in <code>.gitignore</code>	S-2	5 min

Fase 1 — Quick win (1 settimana)

Azione	Riferimento	Sforzo
Ricollegare la pagina Budget (Cash Flow su route propria)	7.1	0,5 g
Reintegrare il client Prima Nota (export, saldi) + fix stato "Approva"	7.2	1 g
Fix card saldo ferie (nome campo) + form monte ferie sull'API esistente	7.8	0,5 g
Indici su <code>journal_entries</code> e affini	P-ARC-5	0,5 g
Rate limiting sul login + configurazione Upstash	S-3	0,5 g
Controllo ruolo sulle API in lettura (middleware + <code>requireRole</code>)	S-4	1 g
Rimozione tab 404, stub e filtri morti in Fatture	7.5	0,5 g

Fase 2 — Consolidamento (2–3 settimane)

Azione	Riferimento	Sforzo
Migrazione storage su Supabase Storage (4 punti di upload)	P-ARC-3	2–3 g
Fix cifratura: hash di lookup per CF/IBAN, fix matcher fornitori, decrypt sugli include	P-ARC-1	2–3 g
Tre fix timbratura (timestamp offline, fallback sede, geofencing server-side)	7.7	2–3 g
Notifiche: campanella in-app → push FCM → cron reminder → email inviti	7.9	3–4 g
Soft delete reale sulle scritture contabili	P-ARC-6	1–2 g
Pulizia API orfane/duplicate + convenzioni in CLAUDE.md	P-ARC-7	1–2 g
Decisione multi-tenancy (congelare o completare)	P-ARC-2	0,5–1 g

Fase 3 — Completamento funzionale (1–2 mesi, in parallelo al consolidamento)

Azione	Riferimento	Sforzo
Service layer per la logica contabile + test (estrazione incrementale)	P-ARC-4	continuativo
Motore regole scadenziario (o rimozione della tab)	7.4	2–3 g
Vincoli relazionali mancanti nel solver + test del solver	7.6	3–5 g
Provider SDI per sync automatico + conservazione — A-Cube validato in sandbox	8.1, 8.5	1 settimana
Open Banking — A-Cube/Salt Edge validato in sandbox	8.2, 8.5	1 settimana
Recupero cedolini non abbinati + UI riassegnazione	7.10	1–2 g
Adeguamento privacy/GDPR (informative, art. 4 St. Lav. per la timbratura)	P-DOC-1	consulenza esterna
Le 5 feature pianificate in <code>modifiche/</code> (inbox avvisi, pagamento personale...)	§3.1	da stimare

Conclusione. Il progetto non ha bisogno di ripartire da capo né di cambiare tecnologie: ha bisogno di una fase disciplinata di consolidamento. Le Fasi 0–2 (circa 3–4 settimane di lavoro effettivo) trasformano un sistema "80% costruito" in un sistema "80% utilizzabile in sicurezza". Solo dopo conviene investire sulle integrazioni esterne (SDI, Open Banking) — comprandole, non costruendole.